

Jillian Johnson

Professor Ren

FIN 4820

May 6, 2026

Honors Project: ETF Simulation

In this project, I replicated the performance of the Pacer Trendpilot US Large Cap ETF using python and data from Yahoo Finance. This ETF tracks the Pacer Trendpilot Index, which uses a rules-based methodology and changes exposure based on three main indicators. Its goal is to maximize growth when the market is trending upward and reduce negative effects of short and long-term downtrends. When the Equity, 50/50, or T-Bill Indicators are triggered, exposure of the index is set to 100% S&P 500, 50% S&P 500 and 50% 3-Month US T-Bills, or 100% 3-Month US T-Bills respectively. There is also a Special Indicator that is essentially a forced 50/50 condition. The criteria to trigger each of these indicators involve the 200-day moving average of S&P 500. Transitions occur at the close of business on the first day following the indicator, and the new position is effective on the second. The transitions follow a strict path of indicators through the sequence: Equity to 50/50 to T-Bill. Additionally, the ETF has an annual fund operating expense of 0.60%.

Methodology

My typical approach to completing projects is starting with what I know. This assignment, however, relied heavily on understanding information I wasn't yet familiar with to even know where to start. So, before I began coding, I devoted time to researching how the Pacer ETF worked as well as the data I would need to replicate it. The most helpful document was the Summary Prospectus. It went into sufficient detail about the ETF and its structure. After I understood the content, my usual cycle for complete projects began. It includes coding what I'm already comfortable with, conducting research to fill in gaps, altering and adding to my code, double checking that my code produces what the assignment requires with an AI, and making final edits. I go through this sequence for each part of code.

I split this project into explicit parts, like past assignments, so I could go through them step by step. First, I imported necessary libraries and downloaded required data. I then calculated the returns of S&P 500 Total Return (^SP500TR) and 13 Week Treasury Bill (^IRX) and added them into a new, separate DataFrame that would only contain information most relevant to my analysis. Afterwards, I generated indicator triggers, determined the appropriate indicator for each entry, then assigned the proper weights to ETF components based on the indicator and calculated the ETF's return. A new column was added to the DataFrame for each of these calculations for continuity, visibility, and use in further calculations. Next, I split the data into an evaluation

period using the start date of PTLC data that would be used in the remaining calculations and visualization. This was part of the solution to a challenge I faced, which I will discuss later. After the data was split, I calculated the cumulative performance of my replica, the actual, and the S&P 500 that served as a benchmark. I was then able to move forward to analysis. I created a yearly returns DataFrame to be plotted in a bar chart. Finally, I created a comparative line chart for time series analysis.

Challenges

Although I approached the project in sections so it would go more smoothly, I still encountered some challenges. Initially, the start date of the downloaded data was the date where data collection commenced for the Pacer ETF (PTLC) in Yahoo Finance. Because of this, the column I created to record the 200-day moving average produced NaN values in many entries that I planned to use in my analysis. My solution for this was to choose an earlier start date for the `^SP500TR` and `^IRX`, 01/01/2010, and create a new DataFrame from a split evaluation period lining up with the commencement of PTLC as mentioned above. This evaluation DataFrame was used for remaining calculations and visualization.

Something else I encountered was the difficulty to account for the strict indicator transition path while keeping code concise. I began with using if and else-if statements nested in a loop to determine index exposure; however, I felt it decreased my codes readability. I consulted ChatGPT on alternate ways to complete this action and discovered the functions `np.select` and `np.where`. I did additional research to learn more about them and implemented the functions into my code. Although they did condense and clean up my code, I couldn't figure out how to account for the specific transition criteria while using them. Ultimately, I decided to combine both versions of code using if statements and `np.where`.

Results

My replica of the Pacer Trendpilot US Large Cap ETF performs very similarly year-to-year as the actual ETF. The biggest discrepancies are in 2018, 2020, and 2022. This reveals the pattern of my replica underperforming in bear markets. My model tends to exit slower during times of decreasing stock market values and economic uncertainty. Additionally, my replica follows the cumulative growth trend of the actual ETF's performance very closely; however, it does seem to slightly underperform. I believe the discrepancy is due to the nuance of the 50/50 Indicator. The ETF's Summary Prospectus states, "following the effectiveness of the 50/50 Indicator, the exposure of the Index may be greater than or less than 50% with respect to the S&P 500 and 3-Month US Treasury bills depending on their respective performance until either the Equity Indicator or T-Bill Indicator (described below) is triggered." The index adopts a dynamic exposure, but mine is fixed and always 50% S&P and 50% T-Bill when the 50/50 Indicator is triggered.